

Nile University (NU) IoT Research Internship Report

Day 4

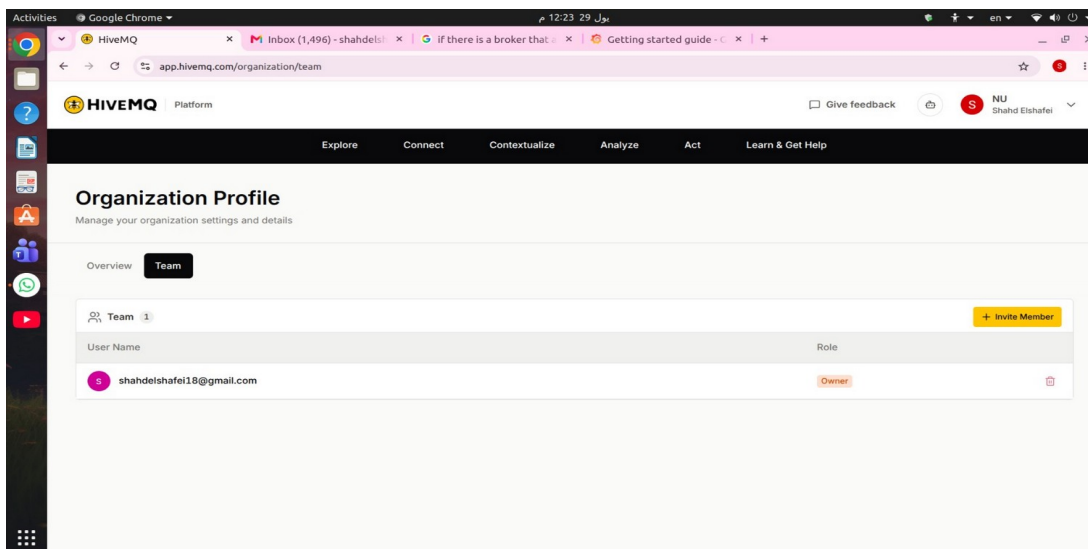
Overview

Day 4 focused on completing a full IoT data pipeline by connecting the existing sensor system to a cloud-based time-series database and visualization platform. Adafruit IO was replaced with HiveMQ as the MQTT broker, and InfluxDB with Grafana was added for time-series storage and real-time visualization.

Task 1: HiveMQ Cloud MQTT Broker Setup

Created a free-tier account on HiveMQ Cloud and set up an MQTT broker instance, obtaining the broker URL, port (8883), username, and password. TLS/SSL encryption was configured for secure communication, and HiveMQ replaced Adafruit IO as the MQTT service.

Connection to Day 3: Day 3 introduced MQTT using Adafruit IO as the broker. This task kept the same MQTT approach but moved to a dedicated, production-style cloud broker (HiveMQ) with proper encrypted communication, which is a more scalable setup than the beginner-oriented Adafruit IO service.



Task 2: InfluxDB Cloud Setup

Created a free-tier account on InfluxDB Cloud, set up an organization and a bucket for data storage, and generated an API token for authentication. The Python script was configured with the InfluxDB URL and credentials to enable writing sensor data to it.

Improvement over Task 1: Task 1 solved the messaging side (getting data published reliably), but MQTT messages are transient and not stored for later analysis. This task added a proper time-series database as a permanent home for the sensor data, which was still missing from the pipeline at this point.

Free #1

Get Started

Overview

Access Management

Test your connection

Topic Subscriptions 1

Unsubscribe from all topics

Subscribe to topics to receive messages from the HiveMQ cluster. You can also set the Quality of Service (QoS) for each topic. The higher the QoS, the more reliable the message delivery is. You can always subscribe to the # wildcard to receive all messages. Please note that the messages from internal probe topics are not displayed here.

Topic	QoS	Actions
#	QoS: 0	

4927	Topic: sensor/distance	QoS: 0	65.476845
4928	Topic: sensor/distance	QoS: 0	65.476845
4929	Topic: sensor/distance	QoS: 0	67.879665
4930	Topic: sensor/distance	QoS: 0	67.879665
4931	Topic: sensor/distance	QoS: 0	65.476845
4932	Topic: sensor/distance	QoS: 0	65.476845

Topic	QoS	Actions
 #	QoS: 0	

Task 3: Python Script Modifications

Improvement over Task 2: Task 2 only set up the InfluxDB account and bucket; this task made the actual connection live by updating the code so real sensor readings flow into both HiveMQ (for real-time messaging) and InfluxDB (for storage) at the same time.

Task 4: Grafana Cloud Setup and Visualization

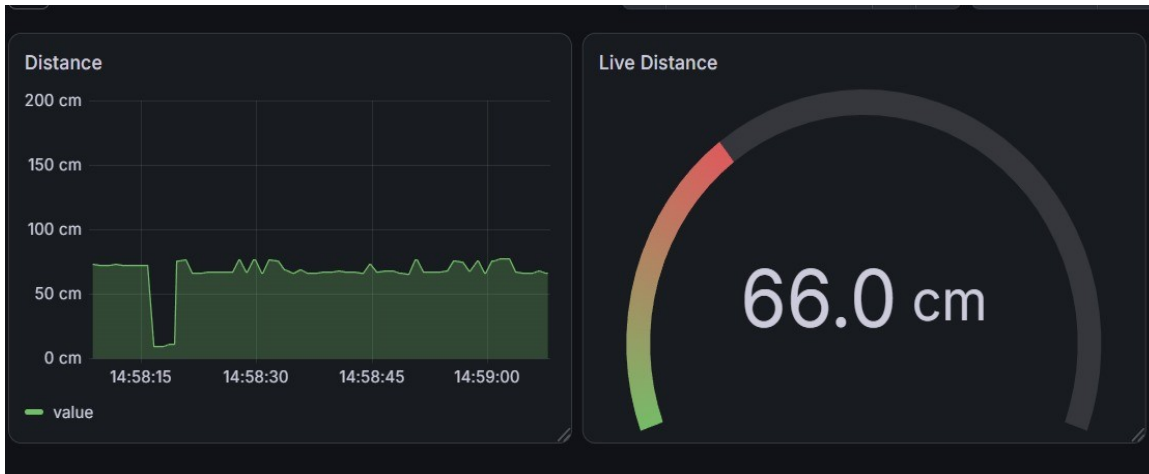
Improvement over Task 3: Task 3 got the data into InfluxDB, but stored data on its own is not easy to interpret. This task closed that gap by turning the stored time-series data into a live, readable dashboard, similar in purpose to the local visualization done with matplotlib on Day 2, but now

cloud-based and continuously updating.

Task 5: Integration and Testing

Started the full IPC system (sensor → MQTT → LED) and verified the complete data flow: Sensor → Queue → MQTT → InfluxDB → Grafana. Confirmed real-time visualization on the Grafana dashboard and verified that LED control still worked correctly through MQTT messages.

Improvement over Task 4: each previous task built one piece of the pipeline in isolation. This task validated that all the pieces worked together correctly end to end, from hardware sensing through to cloud visualization and remote actuation.



Architecture Summary

The system runs on a Raspberry Pi 4 with three cooperating processes: `sensor.py` (reads the HC-SR04 sensor), `mqtt.py` (publishes readings and writes to InfluxDB), and `led.py` (controls the LED on GPIO 17). Sensor readings flow through `mqtt.py` to both HiveMQ (MQTT, TLS/SSL) and InfluxDB Cloud (HTTP API). Grafana Cloud queries InfluxDB to render a live dashboard, and control commands sent through the dashboard travel back via MQTT to the LED process.

Key Learnings

Time-series databases:

- InfluxDB stores time-stamped data points inside buckets, grouped under organizations, with tokens used for API authentication

Visualization tools:

- Grafana provides dashboards made of panels (graphs, tables, stats), connected to data sources such as InfluxDB, with configurable auto-refresh for live updates

Cloud MQTT brokers:

- HiveMQ is a cloud-based MQTT broker using TLS/SSL encryption over the standard secure MQTT port 8883, authenticated with broker credentials

Integration Challenges

- Getting the correct InfluxDB URL/endpoint format (e.g., <https://us-central1-1.gcp.cloud2.influxdata.com>)
- Ensuring the API token had write access to the correct bucket
- Configuring Grafana with the correct data source URL and credentials
- Setting an appropriate dashboard refresh rate
- Formatting data correctly as JSON/Point structures for InfluxDB writes

Tools & Technologies

- Cloud platforms: HiveMQ, InfluxDB Cloud, Grafana Cloud
- Protocols: MQTT (TLS/SSL), HTTP (InfluxDB API)
- Python libraries: paho-mqtt, influxdb_client, gpiozero
- Database: InfluxDB (time-series)
- Visualization: Grafana
- Hardware: Raspberry Pi 4, HC-SR04 sensor, LED

Outcome

- Replaced Adafruit IO with a dedicated, encrypted cloud MQTT broker (HiveMQ)
- Added persistent time-series storage for sensor data using InfluxDB Cloud
- Connected the existing Python pipeline to write to both the broker and the database
- Built a live Grafana dashboard for real-time visualization of sensor data
- Verified the complete end-to-end pipeline: sensing, messaging, storage, visualization, and remote control all working together